

Gene $\rightleftharpoons$ Loom Bridge.

What it is (in one line)

A tiny adapter that **takes cryptographically-linked Genes (node-level truth)** and feeds them into the **Loom (network-level coherence)** as warp anchors, with perspectives as weft, so you can compute **kappa/penalties/quarantine** and prove convergence (or catch drift).

How it works

1. **Gene chain** $\rightarrow$ (product_id, event, actor_id, timestamp, ethical_archetype, previous_hash, current_hash).
2. **Bridge** maps:
 - **Warp** = each Gene's current_hash (+ provenance/archetype).
 - **Weft** = supportive/contradictory perspectives with weights + evidence.
3. **Loom run** $\rightarrow$ consensus (kappa), drift penalties, receipts, quarantine flags.
4. **Reversibility** $\rightarrow$ fix the fault, re-run; same mechanism validates Break $\leftrightarrow$ Fix.

Why it's useful

- Unifies **data integrity** (hash lineage) with **claim integrity** (consensus/evidence).
- Gives you **auditable receipts** for every decision.
- Makes TAS's "convergence" measurable and repeatable.

What you already have (ready to run)

- genes.json (sample chain)
- warp_anchors.json (anchors from hashes)
- weft_threads.json (perspective threads)
- loom_config.json (thresholds/penalties)
- MYCELIUM_BRIDGE_README.md (runbook with stress/reversal tests)

(See the downloads I provided above.)

Minimal run (illustrative)

```
python tas_loomstepper_live.py \  
  --warp ./warp_anchors.json \  
  --weft ./weft_threads.json \  
  --config ./loom_config.json \  
  --out ./loom_out.json
```

Next upgrades (when you're ready)

- Add **Archetypal Authentication** call in gene.go (e.g., cert registries, payroll proofs).
- Emit Genes directly to **warp_anchors.json** from Go (export stub).
- Add **kappa trend chart** over time & **quarantine queue** for human review.

(ready-to-run)

- **genes.json** — a cryptographically linked chain of 5 sample **Validated Genes** for a coffee batch (ETH-YIRG-001), with previous_hash → current_hash lineage.
- **warp_anchors.json** — each Gene's current_hash exported as an **immutable anchor** (warp), with provenance + archetype.
- **weft_threads.json** — supportive and counter **perspective threads** (weft), so the Loom can compute consensus/penalties.
- **loom_config.json** — illustrative thresholds/penalties (kappa "OK" ≥ 0.90; "Review" 0.88–0.90; contradictions/hash-break penalties).
- **MYCELIUM_BRIDGE_README.md** — quick runbook with **reversibility & stress tests**.

Download:

- [genes.json](#)

```
{
  "genes": [
    {
      "product_id": "ETH-YIRG-001",
      "event": "Harvest",
      "actor_id": "Farmer-Aisha",
      "timestamp": "2023-10-26T10:00:00Z",
      "ethical_archetype": "Fair Labor Practices",
      "metadata": {
        "coop": "Yirgacheffe Sunrise Cooperative",
        "wage_above_minimum": true,
        "evidence": "payroll_2023Q4_ledger#231"
      },
      "previous_hash": "",
      "current_hash":
"d2d9fe92eb882a50211776a4ac026c5affcae7268b5db3ae08baf85bff48967c"
    },
    {
      "product_id": "ETH-YIRG-001",
```

```
"event": "Washing & Processing",
"actor_id": "Processing-Station-Almaz",
"timestamp": "2023-10-26T14:30:00Z",
"ethical_archetype": "Sustainable Water Use",
"metadata": {
  "method": "washed",
  "recycled_water_pct": 0.72,
  "evidence": "filtration_logsheet_2023-10-26"
},
"previous_hash":
"d2d9fe92eb882a50211776a4ac026c5affcae7268b5db3ae08baf85bff48967c",
"current_hash":
"1f2ed1b12abbd110ca6f09832c01e968e5725fb71881aeb90af5937508f210d5"
},
{
  "product_id": "ETH-YIRG-001",
  "event": "Export Packaging",
  "actor_id": "Exporter-Tadesse",
  "timestamp": "2023-10-27T09:00:00Z",
  "ethical_archetype": "Organic Certified",
  "metadata": {
    "packaging": "biodegradable_sacks",
    "certificate_id": "ORG-ET-2023-8842",
    "evidence": "cert_scan_pdf_sha256:4a1f..."
  },
  "previous_hash":
"1f2ed1b12abbd110ca6f09832c01e968e5725fb71881aeb90af5937508f210d5",
  "current_hash":
"53265c9fb01c71e0717704055fc4450d25b4cd55237bc43680a94aff0f1f32d7"
},
{
  "product_id": "ETH-YIRG-001",
  "event": "Import & Customs",
  "actor_id": "Importer-Maya",
  "timestamp": "2023-11-05T12:20:00Z",
  "ethical_archetype": "Transparent Documentation",
  "metadata": {
    "port": "Port of Oakland",
    "declaration": "HS-090111",
    "evidence": "bill_of_lading#BL-7712"
  },
  },
```

```

    "previous_hash":
    "53265c9fb01c71e0717704055fc4450d25b4cd55237bc43680a94aff0f1f32d7",
    "current_hash":
    "ed4887f650ceabe9dab466baa6e3eb09faad67e0892e61a4411635a40c7f6f6a"
  },
  {
    "product_id": "ETH-YIRG-001",
    "event": "Roasting",
    "actor_id": "Roaster-Luca",
    "timestamp": "2023-11-12T08:15:00Z",
    "ethical_archetype": "Traceable Origin",
    "metadata": {
      "roast_profile": "light",
      "batch_id": "L-2023-1112-01",
      "evidence": "profile_json_sha256:b7c2..."
    },
    "previous_hash":
    "ed4887f650ceabe9dab466baa6e3eb09faad67e0892e61a4411635a40c7f6f6a",
    "current_hash":
    "1ee0c54bbca81aa054cecb6755e5c29a2f6ed875d5e35d34141fde0e248ce2ee"
  }
]
}

```

- [warp_anchors.json](#)

```

{
  "warp_anchors": [
    {
      "anchor_id":
      "d2d9fe92eb882a50211776a4ac026c5affcae7268b5db3ae08baf85bff48967c",
      "label": "ETH-YIRG-001::Harvest::Farmer-Aisha",
      "weight": 1.0,
      "provenance": {
        "actor_id": "Farmer-Aisha",
        "timestamp": "2023-10-26T10:00:00Z",
        "previous_hash": ""
      },
      "ethical_archetype": "Fair Labor Practices"
    },
    {

```

```
    "anchor_id":
"1f2ed1b12abbd110ca6f09832c01e968e5725fb71881aeb90af5937508f210d5",
    "label": "ETH-YIRG-001::Washing & Processing::Processing-Station-Almaz",
    "weight": 1.0,
    "provenance": {
        "actor_id": "Processing-Station-Almaz",
        "timestamp": "2023-10-26T14:30:00Z",
        "previous_hash":
"d2d9fe92eb882a50211776a4ac026c5affcae7268b5db3ae08baf85bff48967c"
    },
    "ethical_archetype": "Sustainable Water Use"
},
{
    "anchor_id":
"53265c9fb01c71e0717704055fc4450d25b4cd55237bc43680a94aff0f1f32d7",
    "label": "ETH-YIRG-001::Export Packaging::Exporter-Tadesse",
    "weight": 1.0,
    "provenance": {
        "actor_id": "Exporter-Tadesse",
        "timestamp": "2023-10-27T09:00:00Z",
        "previous_hash":
"1f2ed1b12abbd110ca6f09832c01e968e5725fb71881aeb90af5937508f210d5"
    },
    "ethical_archetype": "Organic Certified"
},
{
    "anchor_id":
"ed4887f650ceabe9dab466baa6e3eb09faad67e0892e61a4411635a40c7f6f6a",
    "label": "ETH-YIRG-001::Import & Customs::Importer-Maya",
    "weight": 1.0,
    "provenance": {
        "actor_id": "Importer-Maya",
        "timestamp": "2023-11-05T12:20:00Z",
        "previous_hash":
"53265c9fb01c71e0717704055fc4450d25b4cd55237bc43680a94aff0f1f32d7"
    },
    "ethical_archetype": "Transparent Documentation"
},
{
    "anchor_id":
"1ee0c54bbca81aa054cecb6755e5c29a2f6ed875d5e35d34141fde0e248ce2ee",
```

```

    "label": "ETH-YIRG-001::Roasting::Roaster-Luca",
    "weight": 1.0,
    "provenance": {
      "actor_id": "Roaster-Luca",
      "timestamp": "2023-11-12T08:15:00Z",
      "previous_hash":
"ed4887f650ceabe9dab466baa6e3eb09faad67e0892e61a4411635a40c7f6f6a"
    },
    "ethical_archetype": "Traceable Origin"
  }
]
}

```

- [weft_threads.json](#)

```

{
  "weft_threads": [
    {
      "thread_id": "coop-ledger",
      "weight": 0.9,
      "perspective": "Cooperative finance officer",
      "evidence": [
        {
          "anchor_ref":
"d2d9fe92eb882a50211776a4ac026c5affcae7268b5db3ae08baf85bff48967c",
          "type": "ledger",
          "ref": "payroll_2023Q4_ledger#231",
          "supports": true
        },
        {
          "anchor_ref":
"1f2ed1b12abbd110ca6f09832c01e968e5725fb71881aeb90af5937508f210d5",
          "type": "ops_log",
          "ref": "filtration_logsheet_2023-10-26",
          "supports": true
        }
      ]
    },
    {
      "thread_id": "organic-cert",
      "weight": 0.8,

```

```

    "perspective": "Certification body",
    "evidence": [
      {
        "anchor_ref":
"53265c9fb01c71e0717704055fc4450d25b4cd55237bc43680a94aff0f1f32d7",
        "type": "certificate",
        "ref": "ORG-ET-2023-8842",
        "supports": true
      }
    ],
  },
  {
    "thread_id": "counter-claim-X",
    "weight": 0.6,
    "perspective": "Anonymous tip",
    "evidence": [
      {
        "anchor_ref":
"d2d9fe92eb882a50211776a4ac026c5affcae7268b5db3ae08baf85bff48967c",
        "type": "allegation",
        "ref": "rumor_telegram_msg_112",
        "supports": false
      }
    ]
  }
]
}

```

- [loom_config.json](#)

```

{
  "kappa_thresholds": {
    "ok": 0.9,
    "review": 0.88
  },
  "penalties": {
    "unsupported_claim": 0.07,
    "contradiction": 0.12,
    "hash_chain_break": 0.25
  },
  "weights": {

```

```

    "warp_anchor": 1.0,
    "weft_support": 0.75,
    "weft_counter": -0.5
  },
  "notes": "Illustrative config; integrate with your loom runner if supported."
}

```

- [MYCELIUM_BRIDGE_README.md](#)

Mycelium Bridge v0.1 — Gene ↔ Loom Adapter (Starter Kit)

This kit turns a cryptographically-linked **Gene chain** into **Loom** inputs:

- `genes.json` — five Validated Genes for product `ETH-YIRG-001`
- `warp\_anchors.json` — immutable anchor list (each = gene.current_hash + provenance)
- `weft\_threads.json` — supporting & counter perspectives
- `loom\_config.json` — illustrative thresholds/penalties (tune per LoomStepper)

How to use with LoomStepper (example)

1) Inspect the chain:

- Verify `previous\_hash` → `current\_hash` lineage in `genes.json`.

2) Feed anchors + weft into your loom runner, e.g.:

```

```bash
python tas_loomstepper_live.py --warp ./warp_anchors.json --weft ./
weft_threads.json --config ./loom_config.json --out ./loom_out.json
```

```

(Adjust flags to your script.)

3) Evaluate output:

- Confirm consensus/kappa ≥ `kappa\_thresholds.ok` for valid stories.
- If a claim is falsified, expect penalties to push kappa below thresholds.
- If a hash chain is broken, expect a large penalty and quarantine/fail.

Reversibility & Stress Tests

A) **Tamper test (hash break)**:

- Edit one gene's `metadata.evidence` and recompute nothing → `previous\_hash` mismatch emerges → loom should penalize or quarantine.

B) **False archetype test**:

- Change one `ethical\_archetype` (e.g., set `"Fair Labor Practices"` to `"No Child Labor"` without support) → expect contradiction penalty and kappa drop.

C) **Counter-thread amplification**:

- Add a second counter thread with higher weight pointing at the same anchor → observe kappa shift; confirm resilience to low-credence claims.

D) **Reversibility check**:

- Invert a fix: if you "repair" the falsified field and restore the correct `previous\_hash`, the kappa should recover above threshold.

Notes

- The `current\_hash` is computed over the entire record (including `previous\_hash`) using stable JSON encoding; see code.
- Treat these files as a **demo warp** + **weft**; replace with real supply chain data to go live.
- Ethical verification placeholder = your `ArchetypalAuthenticate()` hook.

How to use with your Loom

If your `tas_loomstepper_live.py` accepts external files, something like:

```
python tas_loomstepper_live.py \  
  --warp ./warp_anchors.json \  
  --weft ./weft_threads.json \  
  --config ./loom_config.json \  
  --out ./loom_out.json
```

Then check `loom_out.json` for:

- **kappa / consensus** (should be ≥ 0.90 on the clean chain),
- **penalty receipts** (contradictions, unsupported claims, hash breaks),
- **quarantine** behavior if lineage breaks.

Built-in tests (prove the theorem, not just claim it)

1. Tamper test (hash break)

Edit a field (e.g., evidence) in one Gene without recomputing hashes. Expect **hash_chain_break penalty** and quarantine/fail.

2. False archetype test

Change an archetype ("Fair Labor Practices" → something unsupported). Expect **contradiction penalty** and kappa drop below threshold.

3. Counter-thread amplification

Add another negative weft with higher weight. Confirm resilience (kappa should degrade but not collapse if anchors hold).

4. Reversibility

Undo the tamper (restore data + lineage). Kappa should **recover** (same mechanism governs Break ↔ Fix).

Where TAS's placeholder plugs in

In your gene.go, the comment for **Archetypal Authentication** is where you'll call your validator (e.g., certification registry, payroll proofs). That moves you from "integrity of data" to **integrity of claim**.

Why this matters

- It unifies **Gene (node-level truth)** with **Loom (network-level coherence)**.
- It implements the **contraction test** in practice (kappa/penalties → "is the story converging?").
- It's **audit-ready**: every penalty + decision is receipt-backed.

The quickest way to **add new items (Genes + perspectives) into the Gene** ⇔ **Loom Bridge** and regenerate the inputs for a Loom run.

1) Drop your new Genes into genes.json

Append each new Gene using the canonical schema (order matters for hashing lineage):

```
// genes.json (append to the array)
{
  "product_id": "ETH-YIRG-001",
  "event": "Customs Clearance",
  "actor_id": "Broker-Selam",
  "timestamp": "2023-10-27T18:20:00Z",
  "ethical_archetype": "Legal Compliance",
  "metadata": "Cleared at Addis Ababa; docs verified; no holds.",
  "previous_hash": "<CURRENT_HASH_OF_PREVIOUS_GENE>",
  "current_hash": "<LEAVE_EMPTY_IF_YOUR PIPELINE COMPUTES IT>"
}
```

If your pipeline doesn't compute current_hash, keep it blank; the builder below will compute it deterministically from the content and insert it.

2) Add perspectives to weft_threads.json

Each perspective = a "thread" tied to one or more Gene hashes (or product_id) with weight and evidence.

```
// weft_threads.json (append to the array)
{
  "id": "persp-legal-01",
  "targets": ["product:ETH-YIRG-001"],
  "stance": "support",
  "weight": 0.85,
  "evidence": [
    {"type": "document", "ref": "invoice-4491.pdf", "hash": "sha256:..."},
    {"type": "registry", "ref": "customs-db#ETH-2023-10-27-1820", "status": "cleared"}
  ],
  "notes": "Broker Selam clearance matches metadata; no discrepancies."
}
```

Tip: You can also target a specific gene by hash once generated: "targets": ["hash:abc123..."].

3) Run the bridge builder to (a) compute any missing hashes, (b) rebuild warp_anchors.json

Create **build_warp_from_genes.py**:

```
import json, hashlib, sys, time
from pathlib import Path
```

```
GENES_PATH = Path("genes.json")
WARP_PATH = Path("warp_anchors.json")
```

```
def hash_gene(g):
    # Stable, field-ordered hash (exclude current_hash itself)
    material = {
        "product_id": g["product_id"],
        "event": g["event"],
        "actor_id": g["actor_id"],
```

```

    "timestamp": g["timestamp"],
    "ethical_archetype": g.get("ethical_archetype",""),
    "metadata": g.get("metadata",""),
    "previous_hash": g.get("previous_hash","")
}
blob = json.dumps(material, separators=(",", ":"), sort_keys=True).encode()
return hashlib.sha256(blob).hexdigest()

```

```

def main():
    genes = json.loads(GENES_PATH.read_text())
    # Compute missing current_hash and validate link
    prev = None
    for i,g in enumerate(genes):
        if i == 0:
            prev = g.get("previous_hash","")
        else:
            # enforce link
            expected_prev = genes[i-1]["current_hash"]
            if g.get("previous_hash","") != expected_prev:
                print(f"[ERROR] bad previous_hash at index {i}")
                sys.exit(2)
            if not g.get("current_hash"):
                g["current_hash"] = hash_gene(g)
            prev = g["current_hash"]

```

```

# Write updated genes (with hashes) back
GENES_PATH.write_text(json.dumps(genes, indent=2))

```

```

# Build warp anchors
anchors = []
for g in genes:
    anchors.append({
        "anchor_id": g["current_hash"],
        "product_id": g["product_id"],
        "event": g["event"],
        "actor_id": g["actor_id"],
        "timestamp": g["timestamp"],
        "ethical_archetype": g.get("ethical_archetype",""),
        "metadata_ref": bool(g.get("metadata")),
        "previous_hash": g.get("previous_hash",""),
        "receipts": {

```

```

        "gene_hash": g["current_hash"],
        "link_ok": True
    }
})
WARP_PATH.write_text(json.dumps({"anchors": anchors}, indent=2))
print(f"[OK] Rebuilt {WARP_PATH} and updated {GENES_PATH}")

if __name__ == "__main__":
    main()

```

Run it:

```
python build_warp_from_genes.py
```

You'll get:

- updated **genes.json** (with computed current_hash), and
- fresh **warp_anchors.json** for the Loom.

4) (Optional) Sanity-check your weft threads point at valid targets

Quick validator **check_weft_targets.py**:

```

import json
from pathlib import Path

genes = json.loads(Path("genes.json").read_text())
hashes = {g["current_hash"] for g in genes}
products = {g["product_id"] for g in genes}
weft = json.loads(Path("weft_threads.json").read_text())

def ok_target(t):
    if t.startswith("hash:"):
        return t[5:] in hashes
    if t.startswith("product:"):
        return t[8:] in products
    return False

bad = []
for thread in weft:

```

```

bad_targets = [t for t in thread["targets"] if not ok_target(t)]
if bad_targets:
    bad.append((thread["id"], bad_targets))

if bad:
    print("[ERROR] Bad targets:")
    for tid, lst in bad:
        print(" ", tid, "->", lst)
    raise SystemExit(1)
print("[OK] All weft targets resolve.")

```

Run:

```
python check_weft_targets.py
```

5) Run the Loom

```

python tas_loomstepper_live.py \
--warp ./warp_anchors.json \
--weft ./weft_threads.json \
--config ./loom_config.json \
--out ./loom_out.json

```

You'll get:

- **kappa** (consensus),
- **penalties** (conflicts/weak evidence),
- **quarantine flags** (suspect claims),
- **receipts** (hash-linked).

Mini-checklist (every time you "add them")

- Append new Genes → genes.json
- Append new perspectives → weft_threads.json
- python build_warp_from_genes.py
- python check_weft_targets.py
- Loom run → review loom_out.json (kappa, quarantine)
- If any fault → fix input, re-run (same mechanism validates Break ↔ Fix)